

Data-Oriented Design (DOD)

Adam HAMOUC

May 2026

1. Intention

Le DOD est une philosophie de conception qui part du hardware plutôt que du domaine métier. Au lieu de se demander "qu'est-ce qu'un Enemy ?", on se demande "quelles données sont traitées ensemble, et comment les organiser pour que le CPU les lise sans cache miss ?".

Contrairement à la POO, le DOD **separe les données des comportements** : des tableaux denses d'un côté, des systèmes qui les traitent en batch de l'autre.

2. Probleme de la POO classique

Deux problèmes de performance fondamentaux :

- **Données éparpillées** : chaque objet est alloué séparément sur le heap. Chaque accès = potentiel cache miss. Le CPU charge une cache line de 64 octets dont une grande partie est inutile pour le système courant.
- **Vtable** : un appel virtuel nécessite 3 indirections (objet → vptr → vtable → fonction) avant d'exécuter la moindre ligne. Le CPU ne peut pas prédire ni prefetcher.

```
// POO : objet auto-suffisant, mais catastrophique en cache
class Enemy {
    vec3 position; vec3 velocity; float health; Mesh* mesh;
    virtual void update(float dt); // vtable lookup à chaque appel
};
std::vector<Enemy*> enemies; // tableau de pointeurs
for (auto* e : enemies) e->update(dt); // cache miss + vtable * 10 000
```

	POO	DOD
Cache miss / frame (10k entites)	~20 000	~2 000
Cycles perdus / frame	~6 000 000	negligeable
Appel de systeme	virtuel (vtable)	direct

3. La reponse DOD : SoA + Systemes

```
// SoA : donnees groupees par type, contiguës, zéro gaspillage
struct EnemyStorage {
    std::vector<vec3> positions;
    std::vector<vec3> velocities;
    std::vector<float> healths;
    std::vector<bool> alives;
    size_t count = 0;
};

// Systeme : traite toutes les entites en batch, pas de vtable
void movementSystem(EnemyStorage& s, float dt) {
    for (size_t i = 0; i < s.count; i++)
        s.positions[i] += s.velocities[i] * dt;
    // Cache line = 5 positions consecutives, tout est utilise
}

void healthSystem(EnemyStorage& s, float dmg) {
    for (size_t i = 0; i < s.count; i++)
```

```

        if (s.alives[i]) s.healths[i] -= dmg;
    }

```

4. Packed Array : tableaux toujours denses

Quand une entite est supprimee, on evite les trous dans le tableau avec un **swap-and-pop** $O(1)$. Le tableau reste dense, aucune iteration sur des entites mortes.

```

void removeEnemy(EntityStorage& s, size_t idx) {
    size_t last = s.count - 1;
    std::swap(s.positions[idx], s.positions[last]);
    std::swap(s.velocities[idx], s.velocities[last]);
    std::swap(s.healths[idx], s.healths[last]);
    s.count--;
}

```

5. EntityID et table d'indirection

Apres un swap, les index changent. Le gameplay ne doit jamais manipuler des index directement – il utilise des **EntityID stables**. Une table d'indirection fait le lien en interne.

```

// idToIndex : ID stable -> index actuel dans le tableau dense
std::unordered_map<EntityID, size_t> idToIndex;
std::vector<EntityID> indexToId;

// Acces gameplay : toujours via ID, jamais via index
float getHealth(EntityStorage& s, EntityID id) {
    return s.healths[idToIndex[id]]; // lookup interne, transparent
}

// Les systemes batch n'utilisent JAMAIS la map
void movementSystem(EntityStorage& s, float dt) {
    for (size_t i = 0; i < s.count; i++) // index direct, map ignoree
        s.positions[i] += s.velocities[i] * dt;
}

```

Regle : iterer sur tout le monde → tableau dense direct, map ignoree. Acceder a une entite precise → map pour trouver l'index, puis tableau.

6. Composants optionnels : Sparse Set vs Archetypes

Les entites n'ont pas toutes les memes composants. Deux architectures principales pour gerer ca :

	Sparse Set	Archetypes
Storage	1 tableau dense par composant	1 bloc SoA par combinaison
Iteration	Intersection des listes	Iteration directe sur blocs
Ajout/retrait composant	$O(1)$	Migration de donnees
Complexite	Faible	Elevee
Perf iteration	Bonne	Maximale
Utilise par	Entt, flecs	Unity DOTS, Unreal Mass

Pour un moteur d'apprentissage : Sparse Set. Plus simple a implementer, perf suffisante. Archetypes quand la perf est critique a tres grande echelle.

7. Application moteur Vulkan

```

// DOD s'applique aussi aux draw calls et aux lights
struct DrawCallStorage {

```

```
std::vector<VkPipeline>      pipelines;
std::vector<VkDescriptorSet> descriptors;
std::vector<VkBuffer>        vertexBuffers;
std::vector<uint32_t>        indexCounts;
};
// Tri par pipeline = iteration sur un seul tableau dense
// Z ro cache miss sur les donnees non pertinentes au tri
```